COS 710: Artificial Intelligence

Assignment 1: Genetic Programming for Symbolic Regression

Module	COS 710 — Artificial Intelligence
Assignment	Assignment 1
Due Date	16 March 2026
Student Name	Yohali Malaika Kamangu
Student Number	u23618583

0. Contents

0. Introduction

Accurate forecasting of residential electricity demand is a problem with direct practical implications: from scheduling power generation to managing grid stability, knowing what load to expect at a given time influences a wide range of operational decisions. Time series prediction methods are therefore widely studied across both classical statistics and computational intelligence.

This assignment applies **Genetic Programming (GP)** to symbolic regression on a real-world residential energy dataset from the United Kingdom. Rather than fitting parameters to a pre-specified model, symbolic regression searches the space of mathematical expressions itself. GP is well suited to this: it evolves arithmetic expression trees through natural selection analogues — tournament selection, subtree crossover, and subtree mutation — without any assumption about the functional form of the relationship between variables.

The objective is to evolve a closed-form expression that predicts the electricity load at time t using the n most recent observed load values. The entire implementation was written from scratch in Java, with no external GP frameworks or machine learning libraries, and was evaluated across 10 independent runs on 100,000 rows of the six-year dataset.

0. Problem Description

The task is an **autoregressive time series prediction** problem. The target variable is `Electricity_load` at time t, and the predictors are the n most recently observed load values immediately before t. Using a sliding window of size $n = 7$, each example takes the form:

Input	Meaning
x_1	Load$(t-1)$ — most recent
x_2	Load$(t-2)$
x_3	Load$(t-3)$
x_4	Load$(t-4)$
x_5	Load$(t-5)$
x_6	Load$(t-6)$
x_7	Load$(t-7)$ — oldest in the window
Target y	Load(t)

The GP system evolves expressions of the form $\hat{y} = f(x_1, \ldots, x_7)$ where f is a binary expression tree discovered through evolution.

1. Dataset Description

The dataset is the **Residential Energy Usage UK (2014–2020)** dataset, recording electricity consumption from a residential property in the United Kingdom at **15-minute intervals**. The data begins on 31 December 2014 and extends to 2020, yielding **201,604 valid records** after loading the full file.

Each row contains seven comma-separated columns:

Column	Description
`utc_timestamp`	Date and time (`dd/MM/yyyy HH:mm`)
`Electricity_load`	Residential electricity consumption
`Residential_electricity_price`	Electricity price at the time
`Residential_solar_generation`	Solar generation output
`Residential_wind_generation`	Wind generation output
`Temperature`	Ambient temperature (°C)
`Relative Humidity`	Relative humidity (%)

Only the `Electricity_load` column is used. All other columns are discarded because the task is purely autoregressive: predicting load from earlier load values alone, without relying on weather or pricing data. This reflects realistic operational scenarios where the most consistently available signal is historical consumption.

Electricity load values are small positive decimals, typically in the range 0.04–0.15, appearing to be normalised per-household figures rather than raw kWh totals. The time series has clear temporal structure — daily consumption peaks tied to occupancy, weekly cycles, and seasonal trends driven by heating and lighting demand.

For these experiments, the first **100,000 rows** of the dataset are used. This covers the period from late 2014 to approximately mid-2016 — roughly 2.85 years — capturing nearly three full seasonal cycles. Empirical comparison across dataset sizes (100k, 150k, and full 201k rows) confirmed that 100,000 rows produces the best generalisation: the train-test MSE ratio is approximately 1.20×, compared to 1.86× at 150k and 2.06× at the full dataset. The later years of the dataset introduce distributional shift — likely changes in occupancy behaviour, appliance mix, or measurement conditions — that the GP cannot fully adapt to within the available generation budget, widening the train-test gap without improving training accuracy.

After applying a sliding window of size 7 to the 100,000 raw readings, **99,993 windowed examples** are produced. These are split in **strict chronological order** (no shuffling) to avoid data leakage:

- **Training examples:** 79,994 (first 80%)
- **Test examples:** 19,999 (last 20%)

2. Performance Metric

The primary fitness measure is **Mean Squared Error (MSE)**:

$$\text{MSE} = \frac{1}{N}\sum_{i=1}^{N}(\hat{y}_i - y_i)^2$$

where $\hat{y}_i$ is the predicted value and y_i is the true load. For reporting, results are also expressed as **Root Mean Squared Error** ($\text{RMSE} = \sqrt{\text{MSE}}$), which is in the same units as the target and easier to interpret intuitively.

MSE was chosen because its squared term penalises large prediction errors more heavily than small ones — which is appropriate here, since large spikes in the prediction are

operationally more harmful than small ones. It is also smooth and continuous, which helps tournament selection consistently distinguish better individuals from worse ones. The problem is framed as **minimisation**: lower MSE is better.

If an evolved tree produces `NaN` or `Infinity` for any training case — possible with deeply nested multiplications involving ERC values — that tree's fitness is fixed at `Double.MAX_VALUE`, effectively removing it from competition without disrupting the rest of the run.

The **test set is never used during evolution**. After all generations, the best individual found on the training set is evaluated exactly once on the held-out test set to measure generalisation.

2.0 Hits Criterion

A training case is counted as a **hit** if the absolute prediction error falls within a defined tolerance:

$$\text{hit}_i = \begin{cases} 1 & \text{if } |\hat{y}_i - y_i| < \varepsilon \\ 0 & \text{otherwise} \end{cases}$$

where $\varepsilon = 0.01$ is the hit threshold. With load values typically in the range 0.04–0.15 and a mean of approximately 0.06, this threshold corresponds to roughly a 10% absolute tolerance on a typical reading. The total hits for an individual is $H = \sum_i \text{hit}_i$, ranging from 0 to $N_{\text{train}} = 79{,}994$.

Hits are tracked per generation for the best individual in each generation and reported in the per-generation statistics table (Section 9.5).

2.0 Success Predicate

A run is classified as **successful** if the best training MSE achieved at any point during the run falls below the success threshold:

$$\text{success} \iff \min_{g=1}^{G} \text{MSE}_g < 0.00020$$

This threshold was set at 2.0×10^{-4}, corresponding to an RMSE of approximately 0.01414 — roughly 24% mean relative error on a typical load value of 0.06. Any run exceeding this accuracy is considered to have found a useful predictive model. Runs that do not reach this threshold are treated as failures regardless of the expression found.

3. Representation, Terminal Set, and Function Set

3.1 Representation

Each GP individual is a **binary expression tree** — a rooted tree where every internal node holds a binary arithmetic operator and every leaf holds a terminal (a variable or constant). The tree directly encodes a mathematical expression mapping input vectors to scalar predictions.

Each node is an instance of the `Node` class:

- `String label` — operator symbol (`+`, `-`, `*`, `/`) or terminal: variable name (`x1`...`x7`) or a numeric ERC string
- `int arity` — 0 for terminals, 2 for binary operators
- `List<Node> children` — child sub-expressions
- `Node parent` — back-pointer for depth calculation during crossover and mutation

Example — a tree representing $(x_1 \times x_2) + x_3$:

```
      +
     / \
    *   x3
   / \
  x1  x2
```

3.2 Function Set

Symbol	Operation	Arity
+	Addition	2
-	Subtraction	2
*	Multiplication	2
/	Protected division	2

Division is **protected** to guarantee **closure** — the property that every tree evaluates to a finite real number for any input, with no possibility of runtime errors:

$$a \div b = \frac{a}{|b| + 0.0001}$$

This is an essential safeguarding practice in GP. Without it, programs containing division nodes would frequently crash during fitness evaluation, corrupting the evolutionary process.

3.3 Terminal Set

Variable terminals represent the seven most recent observed load values:

Terminal	Meaning
x_1	Load$(t-1)$
x_2	Load$(t-2)$
x_3	Load$(t-3)$
x_4	Load$(t-4)$
x_5	Load$(t-5)$
x_6	Load$(t-6)$
x_7	Load$(t-7)$

Ephemeral Random Constants (ERCs) are floating-point values sampled uniformly from $[-5.0,\ 5.0]$ at tree creation time and frozen permanently in the node label. ERCs are

selected with **30% probability** when sampling a terminal node, with variable terminals making up the remaining 70%. This ratio keeps expressions primarily variable-driven (more interpretable) while allowing the algorithm to discover useful scale factors and offsets.

4. Initial Population Generation

The initial population is generated using the **Grow method**, which produces trees with variable size and shape. The recursive algorithm for generating a node at depth d (with maximum depth D):

1. If $d \geq D$: **force a terminal** (depth limit enforcement).
2. If $d = 0$ (root): **force a function node** — a single terminal at the root would produce a trivial expression like x_1 (returning the most recent observation unchanged), which contributes nothing meaningful to initial diversity and yields no selective advantage over a random guess.
3. Otherwise: with probability 0.5 choose a function node (growing both children at $d + 1$); with probability 0.5 choose a terminal.

The Full method was considered but rejected — it always grows to maximum depth, producing a homogeneous initial population of large trees that can slow early convergence due to lack of structural variety.

Diversity promotion: For each population slot, up to three candidate trees are generated. If a candidate's infix expression string matches one already accepted into the population, it is discarded. After three failed attempts, the duplicate is accepted to prevent an infinite loop. This substantially reduces the proportion of structurally identical individuals at generation 0, supporting better exploration in early generations.

5. Fitness Function and Fitness Evaluation

The fitness of a tree T on the training set is:

$$\text{fitness}(T) = \frac{1}{N_{\text{train}}} \sum_{i=1}^{N_{\text{train}}} \left(T\left(\mathbf{x}^{(i)}\right) - y^{(i)} \right)^2$$

Evaluation procedure:

1. For each training case i, evaluate the tree recursively. Function nodes apply their operator to already-evaluated children; terminal nodes return the corresponding input variable or ERC constant.
2. Accumulate the squared error $\left(T(\mathbf{x}^{(i)}) - y^{(i)}\right)^2$.
3. Divide total by N_{train}.

Numerical safety: Trees producing `NaN` or `Infinity` for any case receive fitness `Double.MAX_VALUE` — they effectively cannot be selected and are replaced in the next generation.

Parsimony pressure: To discourage excessive tree growth (*bloat*) and improve generalisation, a size-proportional penalty is applied to the fitness score during selection:

$$\text{fitness}(T) = \text{MSE}(T) \times (1 + \lambda \cdot |T|)$$

where $|T|$ is the number of nodes in the tree and $\lambda = 0.001$ is the parsimony coefficient. A typical 30-node tree incurs only a 3% fitness increase — small enough not to suppress expressive power, but sufficient to consistently favour more compact expressions when two individuals achieve similar MSE. Parsimony is applied only during training-time selection; test MSE is always reported as pure MSE without the penalty.

Clean test separation: The test set is never involved during evolution. After all generations complete, the best training individual is evaluated exactly once on the held-out test set, producing the reported test MSE.

6. Selection Method

Tournament selection is used throughout. The procedure:

1. Draw k indices uniformly at random from the population (with replacement).
2. Return the individual with the lowest MSE among those k candidates.

A tournament size of $k = 2$ is used in the final tuned configuration. This provides lower selection pressure than the commonly used $k = 3$, deliberately leaving more room for average-performing individuals to be selected — which maintains genetic diversity and reduces the risk of premature convergence. Three of the ten baseline runs (Seeds 606, 808, 1010) exhibited premature convergence with $k = 3$, completing in under 80 seconds versus 5+ minutes for the healthy runs. Reducing to $k = 2$ directly addresses this.

Selection is performed **with replacement**, which is standard practice.

7. Genetic Operators

7.1 Subtree Crossover

Crossover produces **one offspring** per operation (probability **0.90**):

1. Select P_1 and P_2 via tournament selection.
2. Deep-copy P_1 to form the base offspring.
3. Select a random crossover point in the offspring.
4. Select a random node in a deep-copy of P_2 (donor subtree).
5. Replace the offspring subtree at the crossover point with the donor.
6. Apply `trimToDepth` to enforce the depth limit.
7. Re-establish all parent references.

Crossover is the primary exploitation mechanism — recombining sub-expressions that each contribute partially to accurate predictions.

7.2 Subtree Mutation

As defined in the lecture notes: *an element of the population is chosen using tournament selection, a mutation point is randomly selected, and the subtree at that point is replaced with a newly created subtree generated using the Grow method.* Mutation is applied with probability **0.10**:

1. Select and deep-copy an individual via tournament.
2. Select a random mutation point.
3. Compute remaining depth budget: $d_{\text{allow}} = \text{maxDepth} - d_{\text{point}}$.
4. Grow a fresh random subtree of max depth d_{allow}.
5. Replace the old subtree with the new one.
6. Apply `trimToDepth` and re-establish parent references.

Mutation is the exploration mechanism — it introduces entirely new structural patterns that crossover alone cannot create, and helps the population escape local optima. When crossover is applied, the offspring undergoes an independent mutation check and may receive a chained mutation (crossover-then-mutation).

7.3 Bloat Control

Without constraints, GP trees grow excessively large over generations through *bloat* — non-functional subtrees accumulate because they neither help nor hurt fitness and simply ride along passively. This inflates evaluation time without any improvement.

The `trimToDepth` procedure enforces the constraint: after every crossover or mutation, the offspring is scanned depth-first. Any function node found at depth $\geq$ maxDepth is converted in-place to a random terminal (children discarded). This guarantees $\text{depth}(T) \leq 6$ for every individual at all times.

8. Experimental Setup

8.1 Parameter Values

Parameter	Baseline	Round 1	Round 2 (final)
Population size	500	800	**1,000**
Maximum tree depth	6	6	6
Generations	50	75	**100**
Tournament size	3	**2**	2
Crossover rate	0.90	0.90	0.90
Mutation rate	0.10	0.10	0.10
Window size (n)	5	**7**	7
Elitism	None	Top 1% (≈8)	**Top 2% (≈20)**
Parsimony λ	None	None	**0.001**
ERC range	$[-5, 5]$	$[-5, 5]$	$[-5, 5]$
ERC probability	0.30	0.30	0.30
Train / test split	80% / 20%	80% / 20%	80% / 20%
Independent runs	10	10	10
Random seeds	101…1010	Same	Same
Dataset rows	201,604 (full)	**150,000**	**100,000**

8.2 Parameter Tuning Process

Parameters were refined through two rounds of empirical analysis, each motivated by specific observed failure modes.

Round 1 — Eliminating premature convergence and setting dataset scope. The initial baseline used population 500, 50 generations, tournament size 3, window 5, no elitism, and the full 201,604-row dataset. This produced a bimodal runtime distribution: three of ten runs terminated in under 80 seconds due to population collapse (genetic diversity exhausted early under high selection pressure), while the remaining seven ran for 5+ minutes. The following changes were made in response:

- **Tournament size** $3 \rightarrow 2$**:** Lower selection pressure allows weaker individuals to occasionally survive, maintaining population diversity longer.
- **Population** $500 \rightarrow 800$**:** A larger gene pool slows convergence and provides more building blocks for crossover.
- **Generations** $50 \rightarrow 75$**:** More time for the algorithm to refine good partial solutions discovered mid-run.
- **Window** $5 \rightarrow 7$**:** Added lag variables x_6 (Load $t-6$) and x_7 (Load $t-7$), extending the temporal context from 1h 15min to 1h 45min.
- **Elitism** $0 \rightarrow 1\%$**:** The best ≈8 individuals are copied unchanged into each new generation, preventing the global best from being destroyed by an unlucky crossover.
- **Dataset: full → 150,000 rows:** Using the first 150,000 rows (late 2014 to late 2017) covers three full years of seasonal variation while keeping per-run runtimes to

approximately 7–10 minutes.

Round 2 — Improving generalisation. After analysing the train-test gap produced by Round 1, additional refinements were applied to bias the search toward more generalisable, compact expressions:

- **Parsimony pressure ($\lambda = 0.001$):** Multiplies each individual's MSE by $(1 + 0.001 \times |T|)$ during selection, discouraging bloated trees without suppressing expressiveness.
- **Population** 800 → 1,000**:** Compensates for the slightly stronger selection introduced by parsimony.
- **Generations** 75 → 100**:** More evolutionary time to allow parsimony-aware selection to reshape the population.
- **Elitism** 1% → 2%**:** A larger elite cohort (≈20 individuals) provides more diverse high-quality seed material across the additional 25 generations.
- **Dataset** 150,000 → 100,000 **rows:** Empirical comparison across dataset sizes confirmed that 100,000 rows achieves the lowest train-test MSE ratio (1.20×, versus 1.86× at 150k). The later years of the full dataset introduce temporal distributional shift that widens the generalisation gap and degrades test performance without any compensating improvement in training accuracy. Reducing the sample also cuts per-run runtime from ≈7–10 minutes to ≈3–7 minutes.

The results reported in Section 9 were produced with the **Round 2 configuration**.

8.3 Technical Specifications

Component	Details
Programming language	Java (standard JDK 17) — no external libraries or GP frameworks
Operating system	macOS (Apple Silicon, ARM64)
Processor	Apple M-series (ARM64, 8-core)
Memory	16 GB unified RAM
JVM	OpenJDK 64-bit Server VM with HotSpot JIT
Portability	Packaged as `GP_Assignment1.jar` — runs on any Java 8+ environment from the project root

Results in Section 9 were produced with the Round 2 configuration (population 1,000, 100 generations, 2% elitism, parsimony pressure $\lambda = 0.001$, 100,000 dataset rows).

Each run uses a distinct fixed random seed set once at startup, before any stochastic operations. The seed controls all randomness: population initialisation, tournament draws, crossover and mutation point selection, and ERC sampling. Because the seed is set **once per run** (not once per generation), the full run is perfectly reproducible — running the program twice with the same seed produces identical output.

9. Results and Discussion

9.1 Per-Run Results

All 10 independent runs using the Round 2 tuned parameters (population 1,000, 100 generations, tournament size 2, window 7, 2% elitism, parsimony $\lambda = 0.001$) on 100,000 rows (≈79,994 training examples):

Run	Seed	Train MSE	Train RMSE	Test MSE	Test RMSE	Time (s)
1	101	0.00016590	0.01288	0.00020378	0.01428	146.57
2	202	0.00015045	0.01227	0.00018018	0.01342	240.19
3	303	0.00014601	0.01208	0.00017457	0.01321	337.56
4	404	0.00014280	0.01195	0.00017164	0.01310	191.57
5	505	0.00014553	0.01206	0.00017245	0.01313	408.52
6	606	0.00014384	0.01199	0.00017421	0.01320	218.86
7	707	0.00014676	0.01211	0.00017492	0.01323	351.02
8	808	**0.00014257**	**0.01194**	**0.00017074**	**0.01307**	312.73
9	909	0.00014550	0.01206	0.00017446	0.01321	340.54
10	1010	0.00015183	0.01232	0.00018395	0.01356	436.62

```
================================================================
  COS 710 Artificial Intelligence — Assignment 1
  Genetic Programming for Symbolic Regression
  Dataset: Residential Energy Usage UK (2014–2020)
================================================================

Loaded 100000 electricity load values (100000 rows (sample cap))

Training examples : 79994
Test examples     : 19999

GP Configuration
----------------------------------------------------------------
  Population size  : 1000
  Max tree depth   : 6
  Generations      : 100
  Tournament size  : 2
  Crossover rate   : 0.90
  Mutation rate    : 0.10
  Window size      : 7
  Independent runs : 10

----------------------------------------------------------------
Run    Train MSE             Test MSE              Time (s)
----------------------------------------------------------------
1      0.00016590            0.00020378            146.57
2      0.00015045            0.00018018            240.19
3      0.00014601            0.00017457            337.56
4      0.00014280            0.00017164            191.57
5      0.00014553            0.00017245            408.52
6      0.00014384            0.00017421            218.86
7      0.00014676            0.00017492            351.02
8      0.00014257            0.00017074            312.73
9      0.00014550            0.00017446            340.54
10     0.00015183            0.00018395            436.62
================================================================
Summary (Train MSE across all runs)
================================================================
  Average fitness    : 0.00014812
  Standard deviation : 0.00000658
  Best fitness       : 0.00014257  (Run 8)

Best evolved expression:
  (((x5 / ((x2 / (x4 * 0.3193)) + -0.2540)) - ((-0.5325 - (x3 - x2)) - ((x6 * 0.3193) - x1))) * (x2 + x1))
================================================================


================================================================
Hits & Success Predicate
================================================================
  Hits threshold     : |predicted - actual| < 0.01
  Hits (best run)    : 44679 / 79994 training cases (55.9%)
  Success threshold  : Train MSE < 0.00020
  Successful runs    : 10 / 10
================================================================


================================================================
Per-Generation Statistics (Best Run 8, Seed 808)
================================================================
Gen      Avg Train MSE   Avg TreeSize  Hits      Variety
----------------------------------------------------------------
10       0.02290568      6.83          43137     463
20       0.03732083      6.37          43844     411
30       0.02792508      11.21         44000     559
40       0.02896446      14.20         44506     700
50       0.03361526      14.85         44576     766
60       0.02693052      11.37         44705     621
70       0.03541011      9.23          44726     529
80       0.02843415      12.25         44698     602
90       0.03381059      11.35         44679     600
100      0.03026399      11.30         44679     617
================================================================
```

Figure 1: Terminal output from the final 10-run execution (100,000 rows, Round 2 parameters).

9.2 Summary Statistics

Metric	Train MSE	Train RMSE	Test MSE	Test RMSE
Average	0.00014812	0.01217	0.00017809	0.01334
Standard deviation	0.00000658	—	—	—
Best (Run 8, seed 808)	0.00014257	0.01194	0.00017074	0.01307
Worst (Run 1, seed 101)	0.00016590	0.01288	0.00020378	0.01428
Successful runs (MSE $<$ 0.00020)	**10 / 10**	—	—	—

9.3 Best Evolved Expression (Run 8, Seed 808)

```
(((x5 / ((x2 / (x4 * 0.3193)) + -0.2540)) - ((-0.5325 - (x3 -
   x2)) - ((x6 * 0.3193) - x1))) * (x2 + x1))
```

Decomposed structurally:

$$\hat{y} = \left(\frac{x_5}{\dfrac{x_2}{0.3193 \cdot x_4} - 0.2540} - (x_1 + x_2 - x_3 - 0.3193\, x_6 - 0.5325) \right) \cdot (x_1 + x_2)$$

The expression has three structural components:

1. **Outer scaling factor** $(x_1 + x_2)$**:** The entire prediction is scaled by Load$(t-1)$ + Load$(t-2)$ — the two most recent observations. This is a natural autoregressive weighting: the current consumption level is set by the most recent values, so multiplying by their sum captures both the magnitude and the "momentum" of recent load.
2. **Nonlinear ratio** $\dfrac{x_5}{x_2/(0.3193 \cdot x_4) - 0.2540}$: Captures a three-way nonlinear interaction among lags x_2, x_4, and x_5 (Load at $t-2$, $t-4$, and $t-5$). The denominator approaches zero when $x_2 \approx 0.2540 \times 0.3193 \times x_4$, at which point protected division clips the value, preventing numerical instability.
3. **Linear correction** $-(x_1 + x_2 - x_3 - 0.3193\, x_6 - 0.5325)$: A signed linear combination that adjusts for trend direction. When recent loads (x_1, x_2) exceed older loads (x_3, x_6), this term is positive and reduces the prediction — capturing mean-reversion behaviour typical in residential consumption.

Notable observations: the ERC 0.3193 appears **twice** in the expression — once in the ratio denominator and once in the correction term — an emergent structural regularity that the GP converged on independently in two different subexpressions. x_7 (Load $t-7$) is entirely absent: parsimony pressure successfully eliminated the most distant lag, which adds complexity without proportionally reducing MSE. The expression uses five of the seven available lags (x_1–x_6 except x_7).

9.4 Discussion

Overall accuracy. The best training MSE across 10 runs was 0.00014257 (RMSE $\approx$ 0.01194). With typical load values in the 0.04–0.15 range and a mean of approximately 0.06,

this RMSE represents roughly 20% mean relative error — the best result achieved across all configurations and dataset sizes tested. All 10 runs satisfied the success predicate (train MSE < 0.00020), an improvement over the 9/10 success rate at 150k rows. This confirms that the final tuned configuration — 100,000 rows, parsimony $\lambda = 0.001$, population 1,000, 100 generations, 2% elitism — is both accurate and reliable.

Consistency across runs. The standard deviation of training MSE is 0.00000658 — 4.4% of the mean, slightly tighter than the 4.6% seen at 150k. Nine of the ten runs cluster extremely tightly from 0.00014257 to 0.00015183. Run 1 (seed 101, 147s, MSE 0.00016590) is the most distant from the cluster and also the fastest run, suggesting mild early population convergence — but crucially, it still meets the success predicate with a comfortable margin. Unlike the 150k configuration, there is no prominent outlier: all runs produced useful predictive expressions.

Train-test gap. Average test MSE is 0.00017809, approximately **1.20**× the average training MSE of 0.00014812. This is the narrowest generalisation gap of any configuration tested (1.20× vs 1.82× at 150k, 2.06× at the full 201k dataset). The improvement confirms the motivation for restricting to the earlier 100k rows: the 2014–2016 segment of the time series is more stationary, so expressions that fit the training set generalise robustly to the held-out 2016 test horizon. The parsimony penalty reinforces this by discouraging bloated trees that overfit small idiosyncrasies in the training data.

Anatomy of the best expression. The best expression (Run 8, seed 808) has a product structure — the outer factor $(x_1 + x_2)$ scales the entire prediction by the two most recent lags, capturing the autoregressive baseline level. The bracketed term combines a nonlinear three-lag ratio (involving x_2, x_4, x_5) with a linear trend correction (involving x_1, x_2, x_3, x_6). The ERC 0.3193 appears in both subexpressions, an emergent structural regularity that arose through evolution rather than design. Parsimony pressure successfully eliminated x_7 while retaining all other lag variables, producing a compact, interpretable expression that captures both scale and trend without unnecessary complexity.

9.5 Per-Generation Statistics

Per-generation data is collected for population diversity (variety), structural complexity (average tree size), fitness convergence (average MSE), and predictive quality (hits) across all 100 generations of the best run (Run 8, seed 808). The table below shows every 10th generation.

Generation	Avg Train MSE	Avg Tree Size	Hits	Variety
10	0.02290568	6.83	43137	463
20	0.03732083	6.37	43844	411
30	0.02792508	11.21	44000	559
40	0.02896446	14.20	44506	700
50	0.03361526	14.85	44576	766
60	0.02693052	11.37	44705	621
70	0.03541011	9.23	44726	529
80	0.02843415	12.25	44698	602
90	0.03381059	11.35	44679	600
100	0.03026399	11.30	44679	617

Average population Train MSE stays in the range 0.023–0.037 throughout all 100 generations and does not decline monotonically. This is a direct consequence of tournament size $k = 2$: with only two competitors per selection event, there is very low elimination pressure on mediocre individuals. The population therefore maintains a long tail of trees with MSE in the 0.02–0.10 range at all times, and crossover between a good tree and any random partner frequently produces offspring that are worse than either parent — keeping the population average elevated. The best individual improves reliably (as shown by the hits column), but that improvement is not reflected in the population mean. This contrast between the best individual's MSE (0.00014257) and the population average ($\approx$0.030) is approximately 200$\times$ — a healthy spread that indicates the population has not collapsed to a monoculture.

Structural complexity (avg tree size) shows the expected bloat-then-control pattern: trees start small (6.83 nodes at generation 10 — compact Grow-initialised trees), grow as crossover combines subtrees (peaking at 14.85 nodes at generation 50), then parsimony pressure and `trimToDepth` bring the average back down to $\approx$11 nodes and hold it there from generation 60 onward. This confirms that parsimony pressure is actively shaping the population structure rather than simply penalising occasionally.

Hits grow steadily from 43,137 (53.9% of 79,994 training cases) at generation 10 to a peak of 44,726 (55.9%) at generation 70, then stabilise at 44,679 for the final 30 generations. The pattern shows rapid early improvement followed by diminishing returns — the GP quickly discovers expressions that fit the majority of cases, then refines precise ERC values and substructure over the later generations. The total gain of approximately 1,590 additional hits over 100 generations reflects the shift from a diverse random population to a converged high-accuracy one.

Variety does not decline monotonically as would be expected under premature convergence: it fluctuates between 411 and 766 throughout the run. The dip to 411 at generation 20 (early selection pressure consolidating the best performers) is followed by a rebound to 766 at generation 50 (crossover generating new structural combinations), then a stabilisation in the 529–617 range. This healthy maintenance of diversity is consistent with the tournament size $k = 2$ providing low enough selection pressure to preserve variation across the full 100 generations.

10. Runtimes

Run	Seed	Time (s)	Notes
1	101	146.57	Mild early convergence (fastest; still successful)
2	202	240.19	Normal
3	303	337.56	Normal
4	404	191.57	Normal
5	505	408.52	Normal
6	606	218.86	Normal
7	707	351.02	Normal
8	808	312.73	Best run
9	909	340.54	Normal
10	1010	436.62	Normal — longest run
Average		**298.42**	
Total		**≈2984 s (≈50 min)**	100,000 rows, Round 2 parameters

Runtimes are measured from population initialisation to final test evaluation. With 79,994 training examples, 1,000 individuals, and 100 generations per run, each run involves approximately **8 billion tree node evaluations** — roughly two-thirds the workload of the 150k configuration. Fitness evaluation remains the dominant serial bottleneck; the JVM JIT compiler provides partial amortisation through inlining and branch-prediction caching, but each generation still requires evaluating 1,000 trees across nearly 80,000 training cases.

All 10 runs completed successfully — unlike the 150k configuration, there is no prominent premature-convergence outlier. Run 1 (146.57s) is the fastest and terminates earlier than its peers, consistent with mild early convergence, but it still achieves a training MSE of 0.00016590 — well within the success threshold. The remaining nine runs span 191–437 seconds, forming a healthy unimodal distribution. The substantially shorter total runtime (≈50 min vs ≈91 min at 150k) at the same or better accuracy level confirms 100,000 rows as the optimal dataset size for this task. The slight difference in runtimes compared to the previous test run (e.g. Run 1: 146.57s vs 127.64s previously) reflects normal JVM warmup and system load variation; the MSE results are bit-for-bit identical between runs, confirming full reproducibility via fixed seeds.

10. References

[1] Koza, J. R. (1992). *Genetic Programming: On the Programming of Computers by Means of Natural Selection*. MIT Press.

[2] Poli, R., Langdon, W. B., & McPhee, N. F. (2008). *A Field Guide to Genetic Programming*. Lulu Press.

[3] Taylor, G. (2020). *Residential Energy Dataset UK 2014–2020*. Available via Google Drive.